LLM EVALUATION & BENCHMARKING PLATFORM

Product Requirement Document

(PRD)

Document Version 1.0
September 2026

Prepared by
Monish D — 24BDS1033
Soaham S — 24BDS1021

Project: LLM Evaluation and Benchmarking Platform
Course: Software Engineering — Digital Assignment

Table of Contents

Table of Contents 2
1. Product Vision 3
2. Problem Statement 3
3. Target Users 3
4. Goals and Success Metrics 3
5. Competitive Landscape & Novelty 3
6. Key Features (Prioritized) 4
6.1 Must-Have (MVP) 4
6.2 Should-Have 4
6.3 Could-Have (Future) 4
7. User Experience Principles 4
8. Constraints 5
9. Out of Scope (for MVP) 5
10. Release Plan 5

1. Product Vision

Build a free, self-hostable platform that tells a developer not just "how does Model X score" but "where does Model X actually stand versus the industry-standard models people already trust, and specifically at what — and at what is it weak." Existing public leaderboards give a single aggregate ranking; this product gives a per-capability, positioned, trustworthy comparison, with hallucination/reliability treated as a first-class axis rather than folded into accuracy.

2. Problem Statement

Choosing an LLM today means either trusting a generic leaderboard (which may not reflect your task mix) or manually running your own prompts across models one at a time, with no consistent way to know if a candidate model is genuinely close to a frontier model on the tasks you care about, or just superficially fluent while hallucinating more. There is no free, easy tool that says, in one view: "this model is Strong-tier on coding and math, Weak-tier on long-context and creative writing, and hallucinates 3x more than the baseline on open-domain factual QA."

3. Target Users

- Individual developers and small teams choosing an LLM for a feature, with no budget for enterprise benchmarking tools.
- Students/researchers who need a reproducible academic benchmarking exercise.
- Open-source model builders who want to know how their fine-tune compares to frontier baselines, category by category.

4. Goals and Success Metrics

Goal	Success Metric
Deliver multi-dimensional evaluation	Platform reports quality, hallucination/reliability, capability, efficiency, and cost for every evaluated model in a single run
Deliver industry-relative positioning (novelty)	Every evaluated model receives a Tier (Frontier/Strong/Mid/Weak) per capability category against a documented, versioned baseline set
Zero operating cost	Platform runs fully on free-tier hosting, database, and inference providers with $0 recurring cost
Reproducibility	Any past run can be re-inspected with its exact configuration and baseline version
Usability	A first-time user completes a benchmark run and reads a positioning summary within 10 minutes, unaided

5. Competitive Landscape & Novelty

Public LLM leaderboards and evaluation harnesses (e.g., open evaluation frameworks and hosted arena-style leaderboards) typically provide either (a) a single aggregate score/ranking across a fixed benchmark suite, or (b) raw pass/fail numbers on academic benchmarks without translating them into a plain-language, category-specific "where do you stand" narrative, and rarely treat hallucination as an independently tracked, claim-level metric

alongside task accuracy. Existing evaluation SDKs/frameworks are largely aimed at ML engineers running scripts, not at producing a decision-ready comparative report for a non-specialist.

This project's novelty is the combination of three things that are rarely offered together, for free: (1) automatic normalization of a candidate model's scores against a maintained, versioned set of industry-standard baselines, (2) a per-capability-category tiering and radar profile rather than one aggregate number, and (3) claim-level hallucination/groundedness tracking presented as a first-class reliability axis, not an accuracy sub-component. The build process itself will explicitly research existing open-source benchmarking repos and leaderboard projects (see the Antigravity prompt in this package) to confirm this gap and avoid duplicating existing work, storing findings in a dedicated /reference folder for the team's own use.

6. Key Features (Prioritized)

6.1 Must-Have (MVP)

1. Multi-model, multi-dataset benchmark run configuration and execution.
2. Quality metrics (accuracy/relevance/completeness/instruction-following).
3. Hallucination rate (response-level and, where possible, claim-level) and groundedness.
4. Capability categorization (coding, math, reasoning, creative writing, summarization, factual/grounded QA, long-context).
5. Industry Comparison Engine: baseline registry, normalized scoring, tiering, capability radar, positioning summary.
6. Efficiency (latency, tokens) and cost tracking with a cost-vs-performance view.
7. Comparative dashboard with drill-down into individual responses.
8. Report export (PDF/CSV) and run history.

6.2 Should-Have

- Custom dataset upload.
- Consistency scoring via repeated sampling.
- Historical trend view of a model's tier over time.

6.3 Could-Have (Future)

- Local/self-hosted open-source model evaluation.
- Automated baseline refresh from public sources.
- Model recommendation based on user-declared priorities (e.g., "optimize for lowest cost at Strong-tier coding").

7. User Experience Principles

- Lead with the positioning narrative, not raw numbers — numbers are available on drill-down, but the headline is "Tier + category."
- Never present a single overall "best" model; always show the trade-off (quality vs hallucination vs cost vs latency), matching the source report's explicit design intent.
- Always disclose the baseline's version/date so a positioning claim is never mistaken for a timeless fact.

8. Constraints

- Zero budget: every tool, API, hosting, and database choice must have a genuinely free tier sufficient for an academic-scale project.
- Two-person team, academic timeline.
- Dependent on third-party free-tier availability and rate limits, which can change without notice.

9. Out of Scope (for MVP)

- Fine-tuning or training models.
- Enterprise SSO/multi-tenant billing.
- Guaranteeing baseline scores are always perfectly current with the latest model releases (mitigated by version/date labeling, not solved outright).

10. Release Plan

Aligned to the SDP's bottom-up phased schedule: an internal MVP (P0–P7, backend + evaluation + comparison engine working via API/Postman) precedes the UI milestone (P8–P9), followed by public deployment on Railway (P10) and a validation benchmark run used as the demonstration artifact for the course submission (P11).